NAME: TANKISO MASOEBE

PROGRAMMING WITH C++WEEK 7

1. **Scenario Analysis:**

A payroll calculation function in an IT system is responsible for automatically computing employees' salaries based on predefined rules and employee data. The system collects information such as hours worked, overtime, tax deductions, and employee benefits from a company database. When payroll processing begins, the function calculates the gross salary by multiplying the employee's hourly rate by the number of hours worked. It then adds overtime pay if applicable. After determining the gross pay, the system calculates deductions such as income tax, pension contributions, and medical aid. These deductions are subtracted from the gross salary to produce the employee's net pay. The function also generates payroll reports for management and creates payslips for employees. By automating these calculations, the payroll function reduces human errors, saves time, and ensures that employees are paid accurately and on schedule. It also keeps secure records for auditing and compliance purposes.

2. **Concept Research:**

Pass-by-reference in C++ is a method of passing arguments to a function where the function receives the actual memory address of a variable instead of a copy of its value. This allows the function to directly modify the original variable. It is implemented using the reference symbol (&) in the function parameter. Pass-by-reference is useful when a program needs to change the value of a variable inside a function or when large data structures are passed to avoid unnecessary copying. This approach improves efficiency and reduces memory usage. It is commonly used in functions that return multiple results or modify input variables.

3. **Tool Practice:**

In Visual Studio Code, I wrote a C++ function that uses default parameters to simplify function calls. Default parameters allow a function to automatically use predefined values if the user does not provide arguments. For example, I created a function calculateTotal(int price, int quantity = 1) where the quantity defaults to 1 if it is not specified. While testing the code, I noticed that the function could be called with one or two arguments, making it flexible and easier to use. This feature helps reduce repetitive code and improves readability. Using default parameters is helpful when functions often use the same values.

4. **Application Practice:**

A data validation function in an IT system is designed to check whether user input meets required rules before it is stored in a database. For example, a function called validateUserData() can verify that fields such as username, email address, and password are correctly formatted and not empty. The function may check if the email contains "@", ensure the password meets security requirements, and confirm that numeric fields contain only numbers. If the data does not meet these conditions, the function returns an error message. This helps prevent incorrect or harmful data from entering the system and improves overall data accuracy.

```
#include <iostream>

#include <string>



bool validateUserData(const std::string& username, const std::string& email, int age) {

    // Check if username is empty

    if (username.empty()) {

        std::cout << "Error: Username cannot be empty.\n";

        return false;

    }



    // Check if email contains '@'

    if (email.find('@') == std::string::npos) {

        std::cout << "Error: Invalid email address.\n";

        return false;

    }



    // Check if age is within a valid range

    if (age < 0 || age > 120) {
```

```
        std::cout << "Error: Age must be between 0 and 120.\n";
        return false;
    }

    // All validations passed
    return true;
}

int main() {
    std::string username = "TANKISO";
    std::string email = "tankisomasoebe1@gmail.com";
    int age = 25;

    if (validateUserData(username, email, age)) {
        std::cout << "All data is valid.\n";
    } else {
        std::cout << "Data validation failed.\n";
    }

    return 0;
}
```